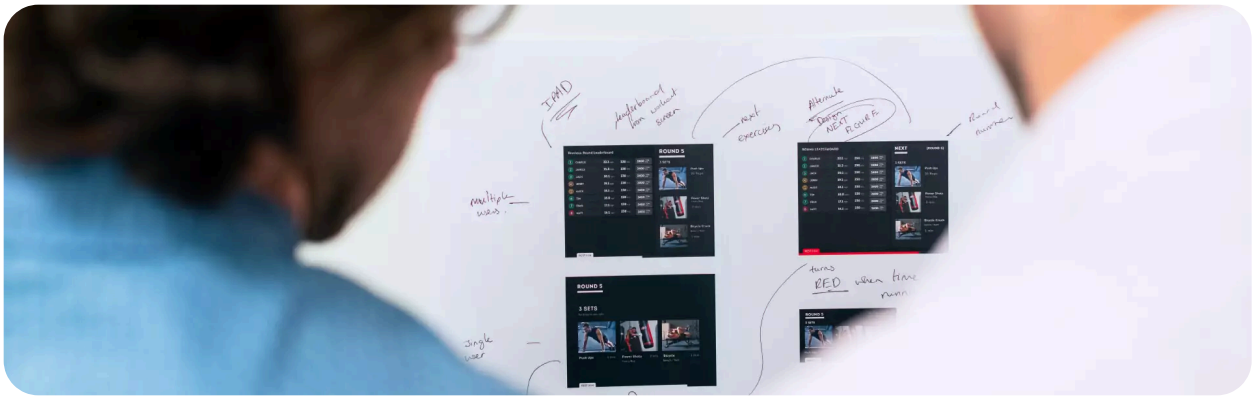
Jelvix

Blog

Article 12 min read Kyrylo Iusov, CPO & Adviser

Software Requirements Specification Example and Guide

Business • Engineering



A software requirements specification (SRS) describes what needs to be done in a project, how it should be done, and to what level the system must conform. This document serves as a point of common understanding for all concerned parties before work on the project is commenced. Without that shared baseline, gaps between intent and execution tend to go unnoticed until they become too costly to ignore.



Stay tuned to the latest industry updates

SUBSCRIBE

TABLE OF CONTENTS

Reasons to use an SRS document

Software Requirements Specification Example: the Structure

1. Introduction
2. Overall description
3. System features and requirements

Techniques and Visual Models

Share

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click Cookie Policy

MANAGE COOKIES REJECT ALL ACCEPT ALL COOKIES

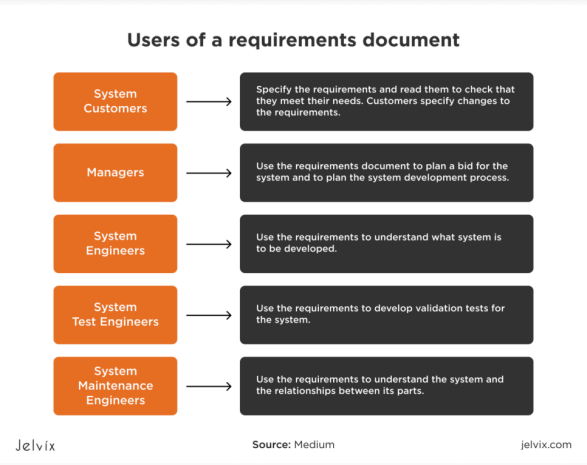
context they are given. Ambiguous requirements produce code that compiles but behaves incorrectly.

- **The core SRS structure has not changed since IEEE 830**. What has changed is how teams use it.
- In 2026, SRS increasingly serves as structured input for AI coding agents, not just a reference document for humans. This approach is called Specification-Driven Development (SDD).
- **For compliance-sensitive products, AI-generated specs and code still require human review.** Regulatory frameworks have not yet caught up with the non-deterministic behavior of AI.



Reasons to use an SRS document

Building new products does not benefit from excessive documentation. However, regardless of which software development methodologies a team follows, SRS should never be omitted. If the crucial aspects of a project are not outlined upfront, too many things can go wrong before anyone catches the drift.



3. Use case diagram
4. Sequence diagram
5. AS-IS and TO-BE process model
6. User Stories
7. Mind Maps

SRS in the Age of AI-Assisted Development

Conclusion

READ NEXT

ENGINEERING | ARTICLE

The Key Difference Between Functional vs. Non-Functional Requirements

HAVE A QUESTION?

BOOK A FREE CONSULTATION

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click Cookie Policy

- **SRS creates the basis for all documentation**. Without it, the entire documentation suite lacks a shared framework. Teams end up with disconnected files that describe the same product differently.
- **SRS sets communication on the right track**. Before the development process commences, product owners, stakeholders, and developers have to agree on the requirements before they turn into code.
- **SRS helps everyone understand the product**. Product owners and developers often hold different visions of the end result. A requirements specification document forces both sides to describe the same system in clear terms.
- **SRS raises development standards**. When everyone understands the product scope and quality benchmarks, those are more likely to be met.
- **SRS surfaces risk early**. Knowing what will be built makes it possible to anticipate what could go wrong, producing more accurate budget and timeline estimates.

A well-written SRS aligns financial and technical goals and holds every party to the same expectations.

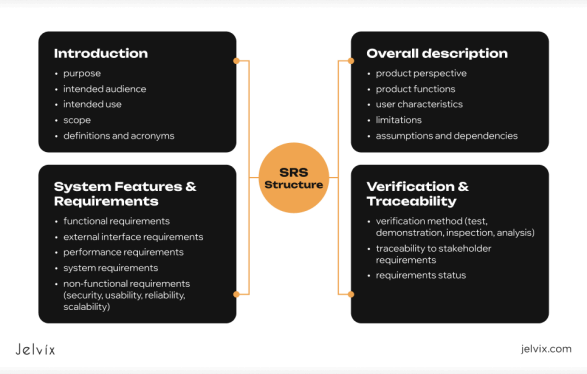
?

Read more about the most common software development strategies and take a look at benefits and drawbacks.

Software Requirements Specification Example: the Structure

An SRS will serve as the team's reference throughout the application development lifecycle. The core functional and technical requirements should be defined before moving to product development, even if design requirements are still being documented in parallel by a separate team. The structure below follows the ISO/IEC/IEEE 29148 standard, which replaced the original IEEE 830 format in 2011 and now serves as the current baseline for SRS documents.

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



In this section, we'll take a look at the structure of the software requirements specification example, describe each section, and its application. You'll see how each fragment of the file comes in handy during the actual project, and what parts are the most important ones.

So, how to prepare an SRS document?

1. Introduction

The introduction frames what the SRS system requirement specification covers, who it is for, and how it is organized.

1.1 Purpose

Describe what the SRS covers and which types of requirements it contains. Keep this to one or two paragraphs.

1.2 Intended audience

Identify the stakeholders, product owners, business analysts, developers, and testers who will work with the document. The full list depends on your team structure and software development methodology.

1.3 Intended use

Outline when the team will consult the SRS: designing new features, planning sprints, estimating costs, evaluating risks, resolving disagreements about scope or quality.

1.4 Scope

Describe the product at a high level: its main role, primary functionality, target users, and key architectural components. This is similar to a stakeholder briefing,

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**

review. Doctors administer treatments and track patient progress remotely. Physical therapy becomes accessible regardless of a patient's proximity to a clinic. A scope section that is specific gives every team member the same mental model of the product before development starts.

1.5 Definitions and acronyms

Define every specialized term, technology, user group, and business entity used in the document. This supports new developer onboarding and eliminates misreadings.

Term	Definition
User	Someone who interacts with the mobile phone application
Admin/ Administrator	System administrator who is given specific permission for managing and controlling the system
Restaurant Owner	Someone who has a restaurant and wants his restaurant to be a part the application
Web-Portal	A web application which presents special facilities for restaurant owner

Jelvix jelvix.com

2. Overall description

The overall description gives an introduction to the functionality of the product, its target users, and the scope of the system without going into the details of an integration. It also frames the software architecture at a level every stakeholder can follow.

2.1 User needs

Provide a list of the issues that the system solves for its users. It will serve as a guide for feature brainstorming, allowing a user experience team to see whether the suggested functions really solve the problem. Make your descriptions general enough to leave room for interpretation when needed.

2.2 Assumptions and dependencies

Assumptions refer to the aspects of the product that will invariably be true. For example, an assumption on a navigation tool designed for driving at night is that users will launch the tool in darkness. Such assumptions determine design without developing an extra feature for daytime.

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**

with smart digital transformation strategies.

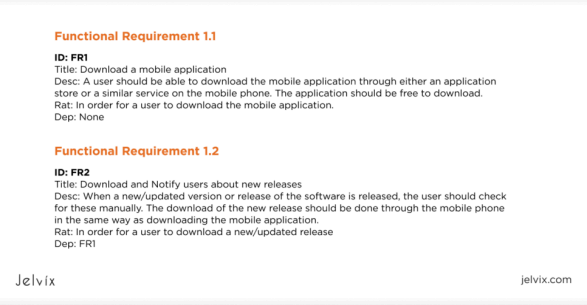
3. System features and requirements

This section contains the detailed functional and non-functional requirements that define how the system behaves and what it must support.

3.1 Functional requirements

Functional requirements answer the question “what will be built?” rather than “how?” or “when?” They describe functionality from the user’s perspective and are ordered by priority. Tech stack specifics are generally left out here because they may shift as the project progresses.

For practical examples of functional requirements and how they differ from non-functional ones, see the detailed guide to functional and non-functional requirements.



3.2 External interface requirements

External interface requirements define the interactions between the system, end users, and external systems, forming the basis of the interface requirements specification. They typically cover four areas: user interfaces, software interfaces, hardware interfaces, and communication interfaces. Design requirements are usually documented separately from backend logic because they are handled by different teams and represent the first stage of development.

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**

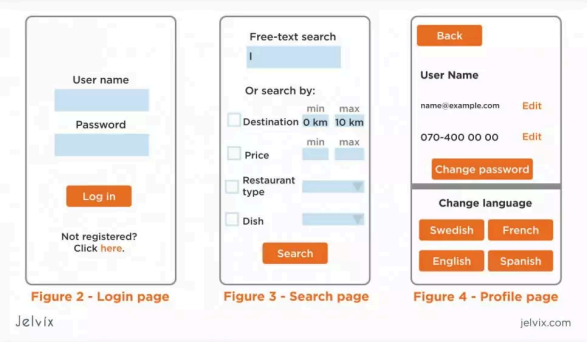


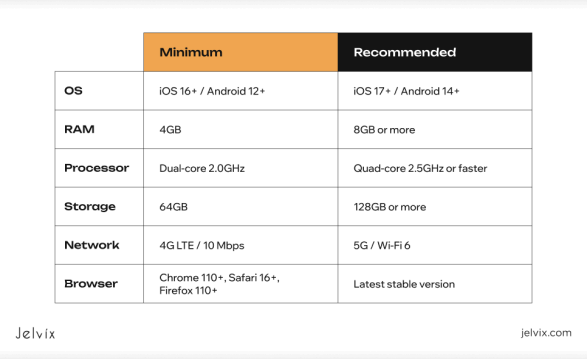
Figure 2 - Login page

Figure 3 - Search page

Figure 4 - Profile page

3.3 System requirements

System requirements define the hardware and infrastructure conditions the product needs to function. These include minimum and maximum thresholds, and for some products, optimal performance values. This is especially important for mobile applications, real-time systems, and anything involving VR/AR or IoT.



	Minimum	Recommended
OS	iOS 16+ / Android 12+	iOS 17+ / Android 14+
RAM	4GB	8GB or more
Processor	Dual-core 2.0GHz	Quad-core 2.5GHz or faster
Storage	64GB	128GB or more
Network	4G LTE / 10 Mbps	5G / Wi-Fi 6
Browser	Chrome 110+, Safari 16+, Firefox 110+	Latest stable version

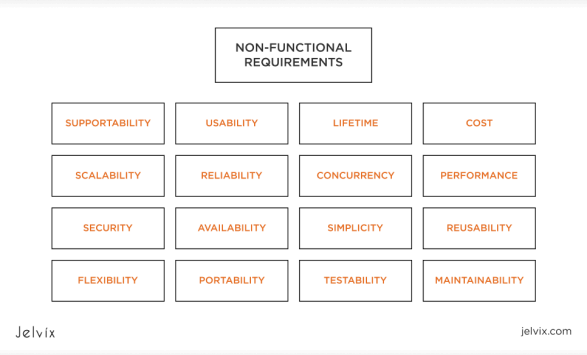
Jelvix jelvix.com

3.4 Non-functional requirements

Non-functional requirements relate to how the system should behave as opposed to what it should do. Performance requirements, security requirements, usability requirements, and scalability requirements are among non-functional requirements.

The difficulty associated with writing non-functional requirements is that such requirements are easy to state. Being fast and being secure have different meanings for different users. Giving numerical values to the requirements helps to make them more understandable.

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



Creating non-functional requirements is difficult for the reason that they are the value. Defining “concurrency” or “portability” is challenging because these terms might mean different things to all participants. This is why we suggest assigning scores to each non-functional requirement. As the project moves along, you can come back to your project requirements and check if the current system responds to initial expectations.

Again, you can take a look at our full guide to non-functional requirements, and review our analysis of existing platforms. We have composed non-functional requirements for popular platforms like Netflix and Instagram – and you can take notions.

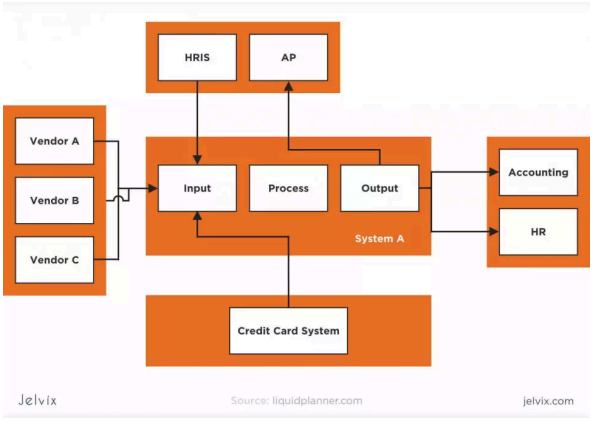
Techniques and Visual Models to Support a Requirements Specification Document

A modern software requirement specification template rarely relies on written descriptions alone. In practice, teams combine visual models, UML diagrams, and Agile artifacts to make requirements easier to validate, communicate, and maintain as the project evolves.

The choice of techniques often depends on the system’s monolith vs microservices architecture, as a distributed system has different documentation needs than a monolithic one. Below are seven techniques teams use most often when structuring an SRS document.

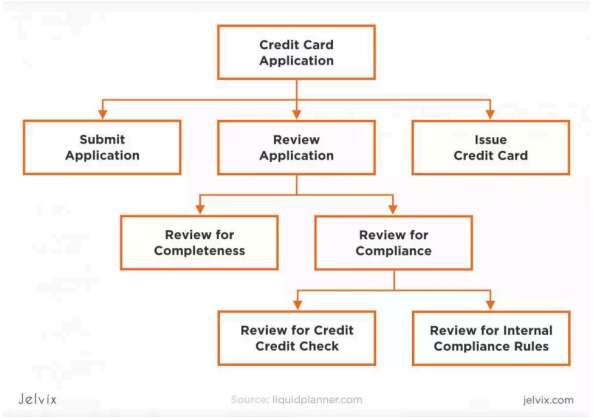
1. Context diagram

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



A context diagram places the system at the center and maps external actors and systems around it, with arrows showing how information flows in and out. It gives developers, designers, and stakeholders a single visual that represents the system as a whole rather than as a list of features.

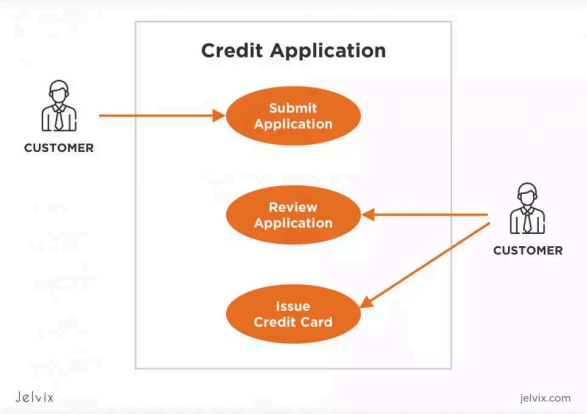
2. Functional decomposition



Functional decomposition involves breaking down the main functions of the system into its structural components. The hierarchy indicates the importance of each feature, either fundamental or subordinate. It is especially important during MVP definition because the first layers of decomposition reveal what should be delivered first.

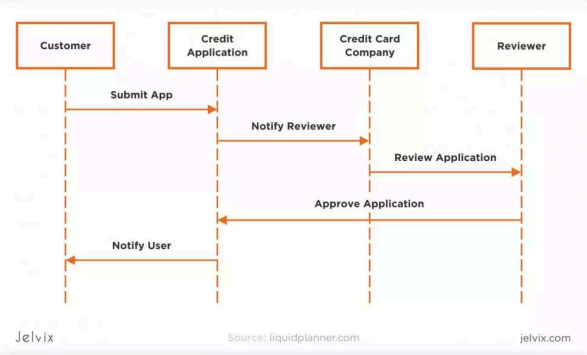
3. Use case diagram

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



While the context diagram depicts various system components, the use case diagram depicts the interaction of the users with those components. Each user type is depicted as an actor, and their paths throughout the system are traced out. This could be placed under user needs or under functional requirements, depending on the audience.

4. Sequence diagram

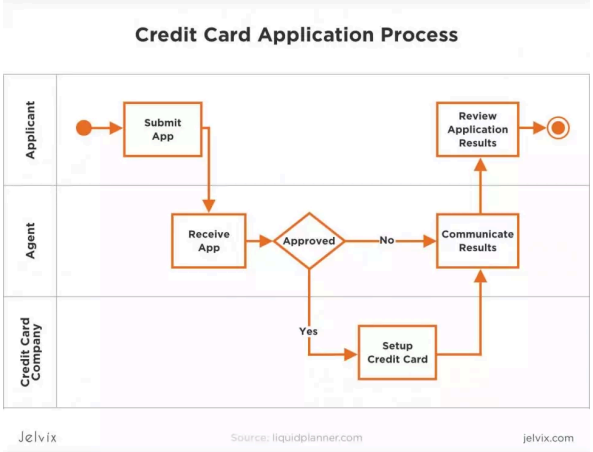


Sequence diagrams provide a representation of how a certain component progresses within the system and what transformation happens at each stage. Sequence diagrams are helpful for representing complex workflows, such as notification triggering, payment authorization sequence, and user onboarding processes, where the order of events matters.

5. AS-IS and TO-BE process model

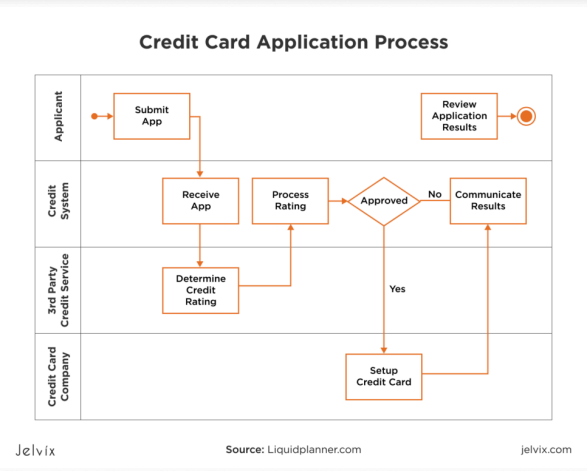
These two diagrams help describe software functionality in relation to business processes. The AS-IS model defines the flow of the process that exists before implementing the solution. It describes the

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



AS-IS models feature applications, agents, and connected parties. This way, the diagram provides an outlook on users who execute the action, middlemen, and final stakeholders. It can also be used to define connections between various features or functionality and its inputs-outputs.

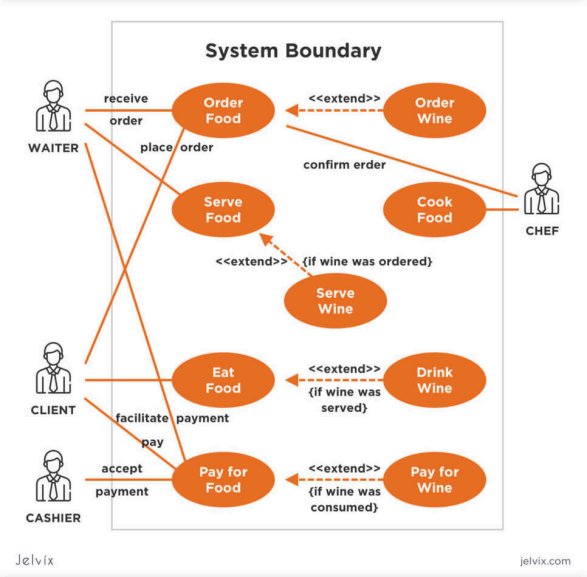
TO-BE process model



The TO-BE model shows how the same process changes after implementing the solution, and identifies the value delivered by the product.

6. User Stories

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



User stories describe actions that a user can perform with the application. You can start with writing epic user stories that refer to general activities under normal conditions. These user stories describe big scenarios that involve a lot of actions.

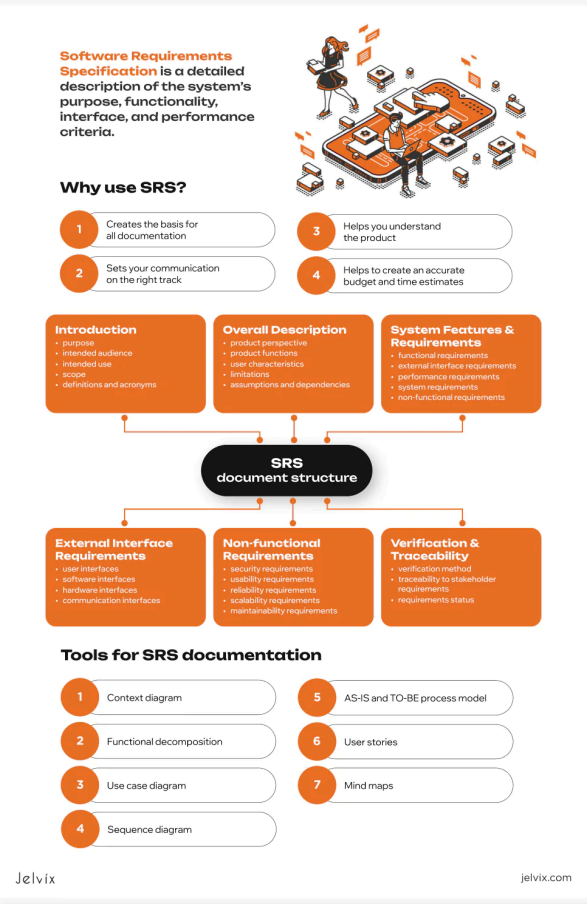
- As a user, I can register an account – it's obvious that we are talking about a multi-step process. Registering an account involves a series of smaller user cases – filling out the form, confirming emails, adding financial information, setting up a profile, etc.
- Epic stories allow developers to see entire blocks of functionality without getting into much detail. Epic stories need to be broken down in the long run, but in the first stages, they help keep the bigger picture and maintain the readability of an SRS document.

Once you have several epic stores, you can break them down to smaller scenarios, using decomposition. If there's a possibility to visualize these scenarios, go ahead and use it.

7. Mind Maps

Mind maps serve as a visual prototype and support collaborative requirement gathering. As they are made in real-time by the entire team, they allow everyone to think in a non-linear manner and make sure that the documentation process does not seem like a formal writing assignment. A mind map for each SRS section can serve as a first draft before prose requirements are written.

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



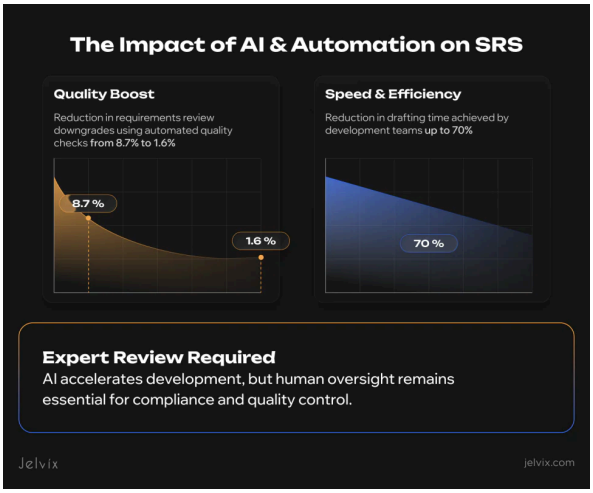
SRS in the Age of AI-Assisted Development

The fundamental structure of an SRS document has not changed since IEEE 830 standardized it. Its role, however, has evolved with the rise of AI requirements engineering and AI-assisted software development.

AI coding agents, such as GitHub Copilot, Cursor, and Claude, use specifications not just as references but as context. In the case where an engineer requests an AI agent to code a certain feature, the quality of the requirements will define the quality of the generated code. A well-structured SRS gives the agent scope boundaries, constraint definitions, and interface expectations. A vague one produces code that compiles but behaves incorrectly.

This shift has accelerated Specification-Driven Development (SDD), where the SRS serves as machine-readable input as much as human documentation. As a result, requirement quality has a more immediate

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



With an increasing involvement of AI in implementation, regulated industries still require expert review of AI-generated requirements and code before production. Current regulatory frameworks have yet to account for the probabilistic nature of large language models.

Teams exploring how this applies to their own custom software development or evaluating AI development services for requirements tooling will find that the fundamentals of a good SRS remain unchanged, but the consequences of writing a poor one are now more immediate.

Conclusion

A software requirements specification aligns technical execution with business goals, helping teams reduce rework, encounter fewer scope disputes, and deliver more predictably.

A well-written SRS also serves as the foundation for AI-assisted workflows. The better the specification, the more effectively AI coding agents can contribute to implementation, and the less human effort is required to correct their outputs.

If your team is working through the structure of a requirements document and wants to see how experienced engineers approach it across different product types, get in touch to discuss practical examples.



This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**

CPO in Jelvix with 8+ years in software development. He has been working as a project manager for more than 6 years. Kyrylo has managed a wide range of projects from various business segments and understands the processes that will drive the product to success. He started from managing small teams, and now he assembles an expert team of more than 40 software developers. His expertise and knowledge of the latest technical innovations have brought Jelvix to the ranks of the most distinguished custom software development companies.

FAQ

What is an SRS document?

What should be included in an SRS document?

How is AI changing how teams write SRS documents in 2026?

How do you distinguish functional and non-functional requirements in an SRS?

You've already rated this article

1 2 3 4 5

Total Rate:
4.5/5

Need a qualified team of developers?

Unlock new business opportunities with the first-rate dedicated development team.

CONTACT US

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**



The Ultimate Guide to the Importance of Requirements Specification



Define Your Project Requirements: 10 Steps

Subscribe to our newsletters

Select topics you are interested in:

All Topics

FinTech

General Tech

HealthTech

Real Estate



Your email address *

SUBSCRIBE

I agree to the Privacy Policy and consent to receive updates based on my selected topics. I can unsubscribe at any time. *

Have a project? Let's get to work!

Full Name *

Email *

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**

I have read and accepted the **Privacy Policy** . *

CONTACT US

Latest In Blog

BUSINESS
How Yardi Integration Cut Manual Data Work by 40+ Hours a Week

BUSINESS | ENGINEERING
Yardi Integration: Where Zapier and Make Hit Their Limits

BUSINESS | ENGINEERING
Why Yardi CRM Integration Breaks and How to Fix It for Good

©2026. Jelvix. All rights reserved.

This website uses cookies for analytics, personalization and advertising. By continuing to browse, you agree to our use of cookies. To learn more, click **Cookie Policy**